

RnD-2: Pyspark как альтернатива быстрому AA-test из H

Контекст и постановка

Цель исследования — понять, как организовать вычисления AA-test на spark-подобном бэкенде так, чтобы сократить время при сохранении корректности метрик.

Вопросы и гипотезы

- H1: наивная схема с многократными проходами по данным масштабируется хуже агрегированного подхода.
- H2: переход к one-pass агрегации уменьшает время и число shuffle-like операций.
- H3: для типичных метрик (mean, var, quantile proxy) достаточно статистик, которые можно собирать ассоциативно.

Методология и план экспериментов

В ноутбуке реализован синтетический spark-like пайплайн: данные разбиваются на партии, после чего сравниваются 3 стратегии:

1. Наивная: отдельный проход на каждую метрику.
2. Batch-агрегация: один проход на набор агрегатов.
3. Incremental combine: партиционные summary + merge.

Сценарии: разные размеры N и число метрик M. Измеряется wall-clock и число проходов.

Результаты

Ключевые числа фиксируются в таблице summary в ноутбуке. Типичный вывод на синтетике: batch/incremental дают ускорение 1.5–3x при росте N и M. Графики времени и проходов см. в разделах "Эксперимент 1" и "Эксперимент 2" ноутбука.

Выводы и рекомендации

- Для AA-test в pyspark-бэкенде важно проектировать API вокруг агрегируемых sufficient statistics.
- Вынести в ядро единый проход по данным с параметризуемым набором статистик.
- Наивные повторные groupBy/agg стоит оставлять только для отладки.

Ссылки

- См. RnD-1: 01_bonferroni_aa_matching/report.md.
- Контекст репозитория: docs/CONTEXT_TZ.md.